

Package ‘BSTFA’

June 2, 2025

Title Bayesian Spatio-Temporal Factor Analysis Model

Description Implements Bayesian spatio-temporal factor analysis models for multivariate data observed across space and time. The package provides tools for model fitting via MCMC, spatial and temporal interpolation, and visualization of latent factors and loadings to support inference and exploration of underlying spatio-temporal patterns. Designed for use in environmental, ecological, or public health applications, with support for posterior prediction and uncertainty quantification. Includes functions such as `BSTFA()` for model fitting and `plot_factor()` to visualize the latent processes.

Version 0.1.0

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

VignetteBuilder knitr

Depends R (>= 2.10)

LinkingTo Rcpp, RcppArmadillo

LazyData true

Imports MASS, RColorBrewer, ggplot2, ggpubr, mgcv, MCMCpack, coda, npreg, matrixcalc, scatterplot3d, sf, Rcpp, lubridate, Matrix, stats, methods, RcppArmadillo

Suggests knitr, rmarkdown, utils, devtools, kableExtra

NeedsCompilation yes

Author Adam Simpson [aut],
Candace Berrett [aut, cre] (ORCID:
<<https://orcid.org/0000-0003-4721-3065>>)

Maintainer Candace Berrett <cberrett@stat.byu.edu>

Contents

bisquare1d	2
bisquare2d	2
BSTFA	3
BSTFAfull	8
computeLogLik	14
compute_summary	14

convergence_diag	15
defineRegion	16
makeNewS	16
makePredS	17
map_spatial_param	18
plot_annual	19
plot_factor	20
plot_fourier_bases	21
plot_location	22
plot_spatial_param	24
plot_trace	25
predictBSTFA	26
utahDataList	27

Index	28
--------------	-----------

bisquare1d	<i>Bisquare bases for 1-dimensional space</i>
------------	---

Description

Evaluate bisquare bases for 1-dimensional space. Used internally in BSTFA and BSTFAfull.

Usage

```
bisquare1d(locs, knots)
```

Arguments

locs	Matrix of coordinates for observed locations.
knots	A vector of the number of knots for each resolution.

Author(s)

Candace Berrett and Adam Simpson

bisquare2d	<i>Bisquare bases for 2-dimensional space</i>
------------	---

Description

Function to evaluate bisquare bases for 2-dimensional space. Used internally in BSTFA and BSTFAfull.

Usage

```
bisquare2d(locs, knots)
```

Arguments

locs	Matrix of coordinates for observed locations.
knots	A vector of the number of knots for each resolution.

Author(s)

Candace Berrett and Adam Simpson

BSTFA

*Reduced BSTFA function***Description**

This function uses MCMC to draw from posterior distributions of a Bayesian spatio-temporal factor analysis model. All spatial processes use one of Fourier, thin plate spline, or multiresolution basis functions. The temporally-dependent factors use Fourier bases. The default values are chosen to work well for many data sets. Thus, it is possible to use this function using only three arguments: `y`mat, `dates`, and `coords`. The default number of MCMC iterations is 10000 (saving 5000); however, depending on the number of observations and processes modeled, it may need more draws than this to ensure the posterior draws are representative of the entire posterior distribution space.

Usage

```
BSTFA(
  ymat,
  dates,
  coords,
  iters = 10000,
  n.times = nrow(ymat),
  n.locs = ncol(ymat),
  x = NULL,
  mean = FALSE,
  linear = TRUE,
  seasonal = TRUE,
  factors = TRUE,
  n.seasn.knots = min(7, ceiling(length(unique(yday(dates)))/3)),
  spatial.style = "fourier",
  n.spatial.bases = min(8, ceiling(n.locs/3)),
  knot.levels = 2,
  max.knot.dist = mean(dist(coords)),
  premade.knots = NULL,
  plot.knots = FALSE,
  n.factors = min(4, ceiling(n.locs/20)),
  factors.fixed = NULL,
  plot.factors = FALSE,
  load.style = "fourier",
  n.load.bases = min(6, ceiling(dim(coords)[1]/3)),
  freq.lon = 4 * diff(range(coords[, 1])),
  freq.lat = 4 * diff(range(coords[, 2])),
  n.temp.bases = ifelse(floor(n.times * 0.1)%2 == 1, floor(n.times * 0.1) - 1,
    floor(n.times * 0.1)),
  freq.temp = n.times,
  alpha.prec = 1/1e+05,
  tau2.gamma = 2,
  tau2.phi = 1e-07,
```

```

sig2.gamma = 2,
sig2.phi = 1e-05,
sig2 = NULL,
beta = NULL,
xi = NULL,
Fmat = matrix(0, nrow = n.times, ncol = n.factors),
Lambda = matrix(0, nrow = n.locs, n.factors),
thin = 1,
burn = iters * 0.5,
verbose = TRUE,
filename = "BSTFA.Rdata",
save.missing = TRUE,
save.output = FALSE
)

```

Arguments

<code>ymat</code>	Data matrix of size <code>n.times</code> by <code>n.locs</code> . Any missing data should be marked by NA. The model works best if the data are zero-centered for each location.
<code>dates</code>	<code>n.times</code> length vector of class 'Date' corresponding to each date of the observed data. For now, the dates should be regularly spaced (e.g., daily).
<code>coords</code>	<code>n.locs</code> by 2 matrix or data frame of coordinates for the locations of the observed data. If using longitude and latitude, longitude is assumed to be the first coordinate.
<code>iters</code>	Number of MCMC iterations to draw. Default value is 10000. Function only saves $(iters - burn) / thin$ drawn values.
<code>n.times</code>	Number of observations for each location. Default is <code>nrow(ymat)</code> .
<code>n.locs</code>	Number of observed locations. Default is <code>ncol(ymat)</code> .
<code>x</code>	Optional <code>n.locs</code> by <code>p</code> matrix of covariates for each location. If there are no covariates, set to NULL (default).
<code>mean</code>	Logical scalar. If TRUE, the model will fit a spatially-dependent mean for each location. Otherwise, the model will assume the means are zero at each location (default).
<code>linear</code>	Logical scalar. If TRUE (default), the model will fit a spatially-dependent linear increase/decrease (or slope) in time. Otherwise, the model will assume a zero change in slope across time.
<code>seasonal</code>	Logical scalar. If TRUE (default), the model will use circular b-splines to model a spatially-dependent annual process. Otherwise, the model will assume there is no seasonal (annual) process.
<code>factors</code>	Logical scalar. If TRUE (default), the model will fit a spatio-temporal factor analysis model with temporally-dependent factors and spatially-dependent loadings.
<code>n.seasn.knots</code>	Numeric scalar indicating the number of knots to use for the seasonal basis components. The default value is $\min(7, \text{ceiling}(\text{length}(\text{unique}(\text{yday}(\text{dates}))) / 3))$, where 7 will capture approximately 2 peaks during the year.
<code>spatial.style</code>	Character scalar indicating the style of bases to use for the linear and seasonal components. Style options are 'fourier' (default), 'tps' for thin plate splines, and 'grid' for multiresolution bisquare bases using knots from a grid across the space.

<code>n.spatial.bases</code>	Numeric scalar indicating the number of spatial bases to use when <code>spatial.style</code> is either 'fourier' or 'tps'. Default value is <code>min(8, ceiling(n.locs/3))</code> .
<code>knot.levels</code>	Numeric scalar indicating the number of resolutions to use for when <code>spatial.style='grid'</code> and/or <code>load.style='grid'</code> . Default is 2.
<code>max.knot.dist</code>	Numeric scalar indicating the maximum distance at which a basis value is greater than zero when <code>spatial.style='grid'</code> and/or <code>load.style='grid'</code> . Default value is <code>mean(dist(coords))</code> .
<code>premade.knots</code>	Optional list of length <code>knot.levels</code> with each list element containing a matrix of longitude-latitude coordinates of the knots to use for each resolution when <code>spatial.style='grid'</code> and/or <code>load.style='grid'</code> . Otherwise, when <code>premade.knots = NULL</code> (default), the knots are determined by using the standard multiresolution grids across the space.
<code>plot.knots</code>	Logical scalar indicating whether to plot the knots used when <code>spatial.style='grid'</code> and/or <code>load.style='grid'</code> . Default is FALSE.
<code>n.factors</code>	Numeric scalar indicating how many factors to use in the model. Default is <code>min(4, ceiling(n.locs/20))</code> .
<code>factors.fixed</code>	Numeric vector of length <code>n.factors</code> indicating the locations to use for the fixed loadings. This is needed for model identifiability. If <code>factors.fixed=NULL</code> (default), the code will select locations with less than 20% missing data and that are far apart in the space.
<code>plot.factors</code>	Logical scalar indicating whether to plot the fixed factor locations. Default is FALSE.
<code>load.style</code>	Character scalar indicating the style of spatial bases to use for the spatially-dependent loadings. Options are 'fourier' (default) for the Fourier bases, 'tps' for thin plate splines, and 'grid' for multiresolution bases. This can be the same as or different than <code>spatial.style</code> .
<code>n.load.bases</code>	Numeric scalar indicating the number of bases to use for the spatially-dependent loadings when <code>load.style</code> is either 'fourier' or 'tps'. This can be the same as or different than <code>n.spatial.bases</code> . Default is <code>min(6, ceiling(dim(coords)[1]/3))</code> .
<code>freq.lon</code>	Numeric scalar indicating the frequency to use for the first column of <code>coords</code> (assumed to be longitude) for the Fourier bases when <code>spatial.style='fourier'</code> and/or <code>load.style='fourier'</code> . Default value is <code>4*diff(range(coords[,1]))</code> .
<code>freq.lat</code>	Numeric scalar indicating the frequency to use for the second column of <code>coords</code> (assumed to be latitude) for the Fourier bases when <code>spatial.style='fourier'</code> and/or <code>load.style='fourier'</code> . Default value is <code>4*diff(range(coords[,2]))</code> .
<code>n.temp.bases</code>	Numeric scalar indicating the number of Fourier bases to use for the temporally-dependent factors. The default value is 10% of <code>n.times</code> .
<code>freq.temp</code>	Numeric scalar indicating the frequency to use for the Fourier bases of the temporally-dependent factors. The default value is <code>n.times</code> .
<code>alpha.prec</code>	Numeric scalar indicating the prior precision for all model process coefficients. Default value is <code>1/100000</code> .
<code>tau2.gamma</code>	Numeric scalar indicating the prior shape for the precision of the model coefficients. Default value is 2.
<code>tau2.phi</code>	Numeric scalar indicating the prior rate for the precision of the model coefficients. Default value is <code>1e-07</code> .
<code>sig2.gamma</code>	Numeric scalar indicating the prior shape for the residual precision. Default value is 2.

<code>sig2.phi</code>	Numeric scalar indicating the prior rate for the residual precision. Default value is $1e-05$.
<code>sig2</code>	Numeric scalar indicating the starting value for the residual variance. If NULL (default), the function will select a reasonable starting value.
<code>beta</code>	Numeric vector of length <code>n.locs + p</code> indicating starting values for the slopes. If NULL (default), the function will select reasonable starting values.
<code>xi</code>	Numeric vector of length <code>(n.locs + p)*n.seasn.knots</code> indicating starting values for the coefficients of the seasonal component. If NULL (default), the function will select reasonable starting values.
<code>Fmat</code>	Numeric matrix of size <code>n.times</code> by <code>n.factors</code> indicating starting values for the factors. Default value is to start all factor values at 0.
<code>Lambda</code>	Numeric matrix of size <code>n.locs</code> by <code>n.factors</code> indicating starting values for the loadings. Default value is to start all loadings at 0.
<code>thin</code>	Numeric scalar indicating how many MCMC iterations to thin by. Default value is 1, indicating no thinning.
<code>burn</code>	Numeric scalar indicating how many MCMC iterations to burn before saving. Default value is one-half of <code>iters</code> .
<code>verbose</code>	Logical scalar indicating whether or not to print the status of the MCMC process. If TRUE (default), the function will print every time an additional 10% of the MCMC process is completed.
<code>filename</code>	Character scalar indicating the filename to use to save the MCMC output. Default value is 'BSTFA.Rdata'.
<code>save.missing</code>	Logical scalar indicating whether or not to save the MCMC draws for the missing observations. If TRUE (default), the function will save an additional MCMC object containing the MCMC draws for each missing observation. Use FALSE to save file space and memory.
<code>save.output</code>	Logical scalar indicating whether to save the output object to filename. Default value is FALSE.

Value

A list containing the following elements (any elements that are the same as in the function input are removed here for brevity):

mu An mcmc object of size `draws` by `n.locs` containing posterior draws for the mean of each location. If `mean=FALSE` (default), the values will all be zero.

alpha.mu An mcmc object of size `draws` by `n.spatial.bases + p` containing posterior draws for the coefficients modeling the mean process. If `mean=FALSE` (default), the values will all be zero.

tau2.mu An mcmc object of size `draws` by 1 containing the posterior draws for the variance of the mean process. If `mean=FALSE` (default), the values will all be zero.

beta An mcmc object of size `draws` by `n.locs` containing the posterior draws for the increase/decrease (slope) across time for each location.

alpha.beta An mcmc object of size `draws` by `n.spatial.bases + p` containing posterior draws for the coefficients modeling the slope.

tau2.beta An mcmc object of size `draws` by 1 containing posterior draws of the variance of the slopes.

- xi** An mcmc object of size draws by `n.seasn.knots*n.locs` containing posterior draws for the coefficients of the seasonal process.
- alpha.xi** An mcmc object of size draws by `(n.spatial.bases + p)*n.seasn.knots` containing posterior draws for the coefficients modeling each coefficient of the seasonal process.
- tau2.xi** An mcmc object of size draws by 1 containing posterior draws of the variance of the coefficients of the seasonal process.
- F.tilde** An mcmc object of size draws by `n.times*n.factors` containing posterior draws of the residual factors.
- alphaT** An mcmc object of size draws by `n.factors*n.temp.bases` containing posterior draws of the coefficients for the factor temporally-dependent process.
- Lambda.tilde** An mcmc object of size draws by `n.factors*n.locs` containing posterior draws of the loadings for each location.
- alphaS** An mcmc object of size draws by `n.factors*n.load.bases` containing posterior draws of the coefficients for the loadings spatial process.
- tau2.lambda** An mcmc object of size draws by 1 indicating the residual variance of the loadings spatial process.
- sig2** An mcmc object of size draws by 1 containing posterior draws of the residual variance of the data.
- y.missing** If `save.missing=TRUE`, a matrix of size `sum(missing)` by draws containing posterior draws of the missing observations. Otherwise, the object is `NULL`.
- time.data** A data frame of size `iters` by 6 containing the time it took to sample each parameter for every iteration.
- setup.time** An object containing the time the model setup took.
- model.matrices** A list containing the matrices used for each modeling process. `newS` is the matrix of spatial basis coefficients for the mean, linear, and seasonal process coefficients. `linear.Tsub` is the matrix used to enforce a linear increase/increase (slope) across time. `seasonal.bs.basis` is the matrix containing the circular b-splines of the seasonal process. `confoundingPmat.prime` is the matrix that enforces orthogonality of the factors from the mean, linear, and seasonal processes. `QT` contains the fourier bases used to model the temporal factors. `QS` contains the bases used to model the spatial loadings.
- factors.fixed** A vector of length `n.factors` giving the location indices of the fixed loadings.
- iters** A scalar returning the number of MCMC iterations.
- y** An `n.times*n.locs` vector of the observations.
- missing** A logical vector indicating whether that element's observation was missing or not.
- day** A numeric vector of length `n.times` containing the day of year for each element in the original dates.
- knots.spatial** For `spatial.style='grid'`, a list of length `knot.levels` containing the coordinates for all knots at each resolution.
- knots.load** For `load.style='grid'`, a list of length `knot.levels` containing the coordinates for all knots at each resolution.
- draws** The number of saved MCMC iterations after removing the burn-in and thinning.

Author(s)

Adam Simpson and Candace Berrett

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(yamat=TemperatureVals, dates=Dates, coords=Coords)

## End(Not run)
```

BSTFAfull

Full BSTFA function

Description

This function uses MCMC to draw from posterior distributions of a Bayesian spatio-temporal factor analysis model. The spatial processes for the mean, linear, and seasonal behavior use one of Fourier, thin plate spline, or multiresolution basis functions. The temporal dependence of the factors is modeled using a vector autoregressive model. The spatially-dependent loadings are modeled using a mean-zero Gaussian process with an exponential covariance structure. The default values are chosen to work well for many data sets. Thus, it is possible to use this function using only three arguments: `yamat`, `dates`, and `coords`. The default number of MCMC iterations is 10000 (saving 5000); however, depending on the number of observations and processes modeled, it may need more draws than this to ensure the posterior draws are representative of the entire posterior distribution space.

Usage

```
BSTFAfull(
  yamat,
  dates,
  coords,
  iters = 10000,
  n.times = nrow(yamat),
  n.locs = ncol(yamat),
  x = NULL,
  mean = FALSE,
  linear = TRUE,
  seasonal = TRUE,
  factors = TRUE,
  n.seasn.knots = min(7, ceiling(length(unique(yday(dates)))/3)),
  spatial.style = "grid",
  n.spatial.bases = ceiling(n.locs/2),
  knot.levels = 2,
  max.knot.dist = n.locs * 0.05,
  premade.knots = NULL,
  plot.knots = FALSE,
  freq.lon = 4 * diff(range(coords[, 1])),
  freq.lat = 4 * diff(range(coords[, 2])),
  n.factors = min(4, ceiling(n.locs/20)),
  factors.fixed = NULL,
  plot.factors = FALSE,
  alpha.prec = 1/1e+05,
```



```

tau2.gamma = 2,
tau2.phi = 1e-07,
sig2.gamma = 2,
sig2.phi = 1e-05,
omega.ii.mean = 1,
omega.ii.var = 1,
omega.ij.mean = 0,
omega.ij.var = 2,
S.F = diag(1, n.factors),
nu.F = n.factors,
phi.gamma = 3,
phi.phi = 0.5,
sig2 = NULL,
beta = NULL,
xi = NULL,
Fmat = matrix(0, nrow = n.times, ncol = n.factors),
Omega = diag(1, n.factors),
Sigma.F = diag(1, n.factors),
Lambda = matrix(0, nrow = n.locs, n.factors),
phi.lambda = rep(1, n.factors),
thin = 1,
burn = floor(iters * 0.5),
c.omega = matrix(0.001, n.factors, n.factors),
c.phi.lambda = rep(0.001, n.factors),
adapt.iter = (burn + 10),
adapt.epsilon = 1e-20,
verbose = TRUE,
filename = "STFA.Rdata",
save.missing = TRUE,
save.output = FALSE
)

```

Arguments

<code>ymat</code>	Data matrix of size <code>n.times</code> by <code>n.locs</code> . Any missing data should be marked by NA. The model works best if the data are zero-centered for each location.
<code>dates</code>	<code>n.times</code> length vector of class 'Date' corresponding to each date of the observed data. For now, the dates should be regularly spaced (e.g., daily).
<code>coords</code>	<code>n.locs</code> by 2 matrix or data frame of coordinates for the locations of the observed data. If using longitude and latitude, longitude is assumed to be the first coordinate.
<code>iters</code>	Number of MCMC iterations to draw. Default value is 10000. Function only saves $(iters - burn) / thin$ drawn values.
<code>n.times</code>	Number of observations for each location. Default is <code>nrow(ymat)</code> .
<code>n.locs</code>	Number of observed locations. Default is <code>ncol(ymat)</code> .
<code>x</code>	Optional <code>n.locs</code> by <code>p</code> matrix of covariates for each location. If there are no covariates, set to NULL (default).
<code>mean</code>	Logical scalar. If TRUE, the model will fit a spatially-dependent mean for each location. Otherwise, the model will assume the means are zero at each location (default).

<code>linear</code>	Logical scalar. If TRUE (default), the model will fit a spatially-dependent linear increase/decrease (or slope) in time. Otherwise, the model will assume a zero change in slope across time.
<code>seasonal</code>	Logical scalar. If TRUE (default), the model will use circular b-splines to model a spatially-dependent annual process. Otherwise, the model will assume there is no seasonal (annual) process.
<code>factors</code>	Logical scalar. If TRUE (default), the model will fit a spatio-temporal factor analysis model with temporally-dependent factors and spatially-dependent loadings.
<code>n.seasn.knots</code>	Numeric scalar indicating the number of knots to use for the seasonal basis components. The default value is $\min(7, \text{ceiling}(\text{length}(\text{unique}(\text{lubridate::yday}(\text{dates}))/3))$, where 7 will capture approximately 2 peaks during the year.
<code>spatial.style</code>	Character scalar indicating the style of bases to use for the mean, linear, and seasonal components. Style options are 'fourier', 'tps' for thin plate splines, and 'grid' (default) for multiresolution bisquare bases using knots from a grid across the space.
<code>n.spatial.bases</code>	Numeric scalar indicating the number of spatial bases to use when <code>spatial.style</code> is either 'fourier' or 'tps'. Default value is $\min(8, \text{ceiling}(n.locs/3))$.
<code>knot.levels</code>	Numeric scalar indicating the number of resolutions to use for when <code>spatial.style='grid'</code> . Default is 2.
<code>max.knot.dist</code>	Numeric scalar indicating the maximum distance at which a basis value is greater than zero when <code>spatial.style='grid'</code> . Default value is $\text{mean}(\text{dist}(\text{coords}))$.
<code>premade.knots</code>	Optional list of length <code>knot.levels</code> with each list element containing a matrix of longitude-latitude coordinates of the knots to use for each resolution when <code>spatial.style='grid'</code> . Otherwise, when <code>premade.knots = NULL</code> (default), the knots are determined by using the standard multiresolution grids across the space.
<code>plot.knots</code>	Logical scalar indicating whether to plot the knots used when <code>spatial.style='grid'</code> . Default is FALSE.
<code>freq.lon</code>	Numeric scalar indicating the frequency to use for the first column of <code>coords</code> (assumed to be longitude) for the Fourier bases when <code>spatial.style='fourier'</code> . Default value is $4 \times \text{diff}(\text{range}(\text{coords[,1]}))$.
<code>freq.lat</code>	Numeric scalar indicating the frequency to use for the second column of <code>coords</code> (assumed to be latitude) for the Fourier bases when <code>spatial.style='fourier'</code> . Default value is $4 \times \text{diff}(\text{range}(\text{coords[,2]}))$.
<code>n.factors</code>	Numeric scalar indicating how many factors to use in the model. Default is $\min(4, \text{ceiling}(n.locs/20))$.
<code>factors.fixed</code>	Numeric vector of length <code>n.factors</code> indicating the locations to use for the fixed loadings. This is needed for model identifiability. If <code>factors.fixed=NULL</code> (default), the code will select locations with less than 20% missing data and that are far apart in the space.
<code>plot.factors</code>	Logical scalar indicating whether to plot the fixed factor locations. Default is FALSE.
<code>alpha.prec</code>	Numeric scalar indicating the prior precision for all model process coefficients. Default value is $1/100000$.
<code>tau2.gamma</code>	Numeric scalar indicating the prior shape for the precision of the model coefficients. Default value is 2.

tau2.phi	Numeric scalar indicating the prior rate for the precision of the model coefficients. Default value is $1e-07$.
sig2.gamma	Numeric scalar indicating the prior shape for the residual precision. Default value is 2.
sig2.phi	Numeric scalar indicating the prior rate for the residual precision. Default value is $1e-05$.
omega.ii.mean	Numeric scalar indicating the prior mean for the diagonal elements of the autoregressive correlation matrix of the factors. Default is 1.
omega.ii.var	Numeric scalar indicating the prior variance for the diagonal elements of the autoregressive correlation matrix of the factors. Default is 1.
omega.ij.mean	Numeric scalar indicating the prior mean for the off-diagonal elements of the autoregressive correlation matrix of the factors. Default is 0.
omega.ij.var	Numeric scalar indicating the prior variance for the off-diagonal elements of the autoregressive correlation matrix of the factors. Default is 2.
S.F	Numeric matrix of size <code>n.factors</code> by <code>n.factors</code> indicating the prior residual covariance matrix for the factors. Default is <code>diag(1,n.factors)</code> .
nu.F	Numeric scalar indicating the prior degrees of freedom for the residual covariance matrix of the factors. Default is <code>n.factors</code> ; must be greater than or equal to <code>n.factors</code> .
phi.gamma	Numeric scalar indicating the prior shape of the spatial range parameter for the spatially-dependent loadings. Default value is 3.
phi.phi	Numeric scalar indicating the prior rate of the spatial range parameter for the spatially-dependent loadings. Default is 0.5.
sig2	Numeric scalar indicating the starting value for the residual variance. If NULL (default), the function will select a reasonable starting value.
beta	Numeric vector of length <code>n.locs + p</code> indicating starting values for the slopes. If NULL (default), the function will select reasonable starting values.
xi	Numeric vector of length <code>(n.locs + p)*n.seasn.knots</code> indicating starting values for the coefficients of the seasonal component. If NULL (default), the function will select reasonable starting values.
Fmat	Numeric matrix of size <code>n.times</code> by <code>n.factors</code> indicating starting values for the factors. Default value is to start all factor values at 0.
Omega	Numeric matrix of size <code>n.factors</code> by <code>n.factors</code> indicating the starting value for the autoregressive correlation of the factors. Default value is the identity matrix.
Sigma.F	Numeric matrix of size <code>n.factors</code> by <code>n.factors</code> indicating the starting value for the residual covariance matrix of the factors. Default value is the identity matrix.
Lambda	Numeric matrix of size <code>n.locs</code> by <code>n.factors</code> indicating starting values for the loadings. Default value is to start all loadings at 0.
phi.lambda	Numeric vector of length <code>n.factors</code> indicating the starting values for the spatial range parameters for each loading. Default value is a vector of 1's.
thin	Numeric scalar indicating how many MCMC iterations to thin by. Default value is 1, indicating no thinning.
burn	Numeric scalar indicating how many MCMC iterations to burn before saving. Default value is one-half of <code>iters</code> .

<code>c.omega</code>	Numeric matrix of starting values for the proposal standard deviations (for the Metropolis random walk algorithm) for sampling proposal values of the autoregressive correlation matrix for the factors. Default is <code>matrix(0.001, n.factors, n.factors)</code> .
<code>c.phi.lambda</code>	Numeric vector of starting values for the proposal standard deviations (for the Metropolis random walk algorithm) for sampling proposal values of the range of the spatially-dependent loadings. Default is <code>rep(0.001, n.factors)</code> .
<code>adapt.iter</code>	Numeric scalar indicating the number of iterations to start adjusting the proposal standard deviations for the Metropolis random walk algorithms. Value must be at least 2 larger than burn. Default value is <code>burn+10</code> .
<code>adapt.epsilon</code>	Numeric scalar indicating the small value to add to the proposal standard deviations when using the adaptive Metropolis random walk algorithms. Default is <code>1e-20</code> .
<code>verbose</code>	Logical scalar indicating whether or not to print the status of the MCMC process. If TRUE (default), the function will print every time an additional 10% of the MCMC process is completed.
<code>filename</code>	Character scalar indicating the filename to use to save the MCMC output. Default value is <code>'BSTFA.Rdata'</code> .
<code>save.missing</code>	Logical scalar indicating whether or not to save the MCMC draws for the missing observations. If TRUE (default), the function will save an additional MCMC object containing the MCMC draws for each missing observation. Use FALSE to save file space and memory.
<code>save.output</code>	Logical scalar indicating whether to save the output object to filename. Default value is FALSE.

Value

A list containing the following elements (any elements that are the same as in the function input are removed here for brevity):

mu An mcmc object of size draws by `n.locs` containing posterior draws for the mean of each location. If `mean=FALSE` (default), the values will all be zero.

alpha.mu An mcmc object of size draws by `n.spatial.bases + p` containing posterior draws for the coefficients modeling the mean process. If `mean=FALSE` (default), the values will all be zero.

tau2.mu An mcmc object of size draws by 1 containing the posterior draws for the variance of the mean process. If `mean=FALSE` (default), the values will all be zero.

beta An mcmc object of size draws by `n.locs` containing the posterior draws for the increase/decrease (slope) across time for each location.

alpha.beta An mcmc object of size draws by `n.spatial.bases + p` containing posterior draws for the coefficients modeling the slope.

tau2.beta An mcmc object of size draws by 1 containing posterior draws of the variance of the slopes.

xi An mcmc object of size draws by `n.seasn.knots*n.locs` containing posterior draws for the coefficients of the seasonal process.

alpha.xi An mcmc object of size draws by `(n.spatial.bases + p)*n.seasn.knots` containing posterior draws for the coefficients modeling each coefficient of the seasonal process.

tau2.xi An mcmc object of size draws by 1 containing posterior draws of the variance of the coefficients of the seasonal process.

- F.tilde** An mcmc object of size draws by `n.times*n.factors` containing posterior draws of the residual factors.
- alphaT** An mcmc object of size draws by `n.factors*n.temp.bases` containing posterior draws of the coefficients for the factor temporally-dependent process.
- Lambda.tilde** An mcmc object of size draws by `n.factors*n.locs` containing posterior draws of the loadings for each location.
- alphaS** An mcmc object of size draws by `n.factors*n.load.bases` containing posterior draws of the coefficients for the loadings spatial process.
- tau2.lambda** An mcmc object of size draws by 1 indicating the residual variance of the loadings spatial process.
- sig2** An mcmc object of size draws by 1 containing posterior draws of the residual variance of the data.
- y.missing** If `save.missing=TRUE`, a matrix of size `sum(missing)` by draws containing posterior draws of the missing observations. Otherwise, the object is `NULL`.
- time.data** A data frame of size `iters` by 6 containing the time it took to sample each parameter for every iteration.
- setup.time** An object containing the time the model setup took.
- model.matrices** A list containing the matrices used for each modeling process. `newS` is the matrix of spatial basis coefficients for the mean, linear, and seasonal process coefficients. `linear.Tsub` is the matrix used to enforce a linear increase/increase (slope) across time. `seasonal.bs.basis` is the matrix containing the circular b-splines of the seasonal process. `confoundingPmat.prime` is the matrix that enforces orthogonality of the factors from the mean, linear, and seasonal processes. `QT` contains the fourier bases used to model the temporal factors. `QS` contains the bases used to model the spatial loadings.
- factors.fixed** A vector of length `n.factors` giving the location indices of the fixed loadings.
- iters** A scalar returning the number of MCMC iterations.
- y** An `n.times*n.locs` vector of the observations.
- missing** A logical vector indicating whether that element's observation was missing or not.
- doy** A numeric vector of length `n.times` containing the day of year for each element in the original dates.
- knots.spatial** For `spatial.style='grid'`, a list of length `knot.levels` containing the coordinates for all knots at each resolution.
- draws** The number of saved MCMC iterations after removing the burn-in and thinning.

Author(s)

Candace Berrett and Adam Simpson

Examples

```
## Not run:
#Example below not run; even the ten iterations will take a minute or two to run.
data(utahDataList)
attach(utahDataList)
out <- BSTFAfull(y=TemperatureVals, dates=Dates, coords=Coords, iters=10)

## End(Not run)
```

computeLogLik	<i>Compute log-likelihood</i>
---------------	-------------------------------

Description

Compute log-likelihood

Usage

```
computeLogLik(out, verbose = FALSE, addthin = 1)
```

Arguments

out	Output from BSTFA or BSTFAfull.
verbose	Logical scalar indicating whether to print status of the log-likelihood computation. Default is FALSE.
addthin	Numeric scalar indicating the number of additional draws to thin by to reduce the computation time. Default is 1 (no additional thinning).

Value

A matrix of size `n.times*n.locs` by draws log-likelihood values for each observation and each posterior draw.

Author(s)

Adam Simpson and Candace Berrett

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(yamat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
loglik <- computeLogLik(out, addthin=2)

## End(Not run)
```

compute_summary	<i>Print computation summary</i>
-----------------	----------------------------------

Description

Print computation summary

Usage

```
compute_summary(out)
```

Arguments

out Output from BSTFA or BSTFAfull.

Value

Prints the computation time per iteration for each parameter.

Author(s)

Adam Simpson

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(yamat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
compute_summary(out)

## End(Not run)
```

convergence_diag	<i>Check effective sample size and geweke diagnostic</i>
------------------	--

Description

Check effective sample size and geweke diagnostic

Usage

```
convergence_diag(out, type = "eSS", cutoff = ifelse(type == "eSS", 100, 0.001))
```

Arguments

out Output from BSTFA or BSTFAfull.

type Character specifying which diagnostic to compute. Options are ess and geweke.

cutoff Numeric scalar indicating the cutoff value to flag parameters that haven't converged.

Value

A list containing convergence diagnostic for all parameters.

Author(s)

Adam Simpson

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(y=TemperatureVals, dates=Dates, coords=Coords, iters=100)
convergence_diag(out)

## End(Not run)
```

defineRegion	<i>Define the initial region of interest.</i>
--------------	---

Description

Define the region of interest. Used internally.

Usage

```
defineRegion(out, location)
```

Arguments

out	Output object from BSTFA or BSTFAfull functions.
location	Matrix of coordinates for the new locations.

Author(s)

Candace Berrett and Adam Simpson

makeNewS	<i>Create a matrix of bisquare bases for a set of observed locations.</i>
----------	---

Description

Create a matrix of bisquare bases for a set of observed locations. Used internally in BSTFA and BSTFAfull.

Usage

```
makeNewS(
  coords,
  n.locations,
  knot.levels = 2,
  max.knot.dist = mean(dist(coords)),
  x = NULL,
  premade.knots = NULL,
  plot.knots = TRUE,
  regions = FALSE
)
```


Arguments

<code>coords</code>	Matrix of coordinates for new locations.
<code>n.locations</code>	Number of observed locations.
<code>knot.levels</code>	Number of levels for the knots. Default is 2.
<code>max.knot.dist</code>	Maximum distance between an observation and a knot. Default is <code>mean(dist(coords))</code> .
<code>x</code>	Optional matrix of covariates to include in the model.
<code>premade.knots</code>	Optional list of length <code>knot.levels</code> each list item containing pre-chosen knots for that level.
<code>plot.knots</code>	Logical scalar indicating whether or not to plot the knots. Default is TRUE.
<code>regions</code>	Logical scalar indicating if the space should be divided into multiresolution regions. Default is FALSE.

Author(s)

Candace Berrett and Adam Simpson

makePredS

Create a matrix of bisquare bases for new locations.

Description

Create a matrix of bisquare bases for new locations. Used internally.

Usage

```
makePredS(out, location)
```

Arguments

<code>out</code>	Output object from <code>BSTFA</code> or <code>BSTFAfull</code> functions.
<code>location</code>	Matrix of coordinates for the new locations.

Author(s)

Candace Berrett and Adam Simpson

map_spatial_param	<i>Plot a map of interpolated spatially-dependent parameter values.</i>
-------------------	---

Description

Plot a map of interpolated spatially-dependent parameter values.

Usage

```
map_spatial_param(
  out,
  parameter = "slope",
  loadings = 1,
  type = "mean",
  yearscale = TRUE,
  new_x = NULL,
  ci.level = c(0.025, 0.975),
  fine = 100,
  color.gradient = (grDevices::colorRampPalette(rev(RColorBrewer::brewer.pal(9, name =
    "RdBu"))))(fine),
  with.uncertainty = FALSE,
  map = FALSE,
  state = FALSE,
  location = NULL,
  addthin = 1
)
```

Arguments

out	Output from BSTFA or BSTFAfull.
parameter	One of 'slope' (default), 'loading', or 'mean'.
loadings	If parameter='loading', an integer indicating which factor loading to plot.
type	One of mean (default), median, ub, or lb indicating which summary statistic to plot at each location.
yearscale	If parameter='slope', a logical scalar indicating whether to translate it to a yearly scale (TRUE; default).
new_x	If the original model included covariates x, include the same covariates for prediction location.
ci.level	If type='lb' or 'ub', the percentiles for the posterior interval.
fine	Integer specifying the number of grid points along both the longitude and latitude directions used to interpolate the parameter. The resulting interpolation grid will contain fine*fine total locations. If map=TRUE, state=TRUE, and location is specified, the grid will be clipped to the boundaries of the specified state, removing locations outside of it.
color.gradient	The color palette to use for the plot. Default is colorRampPalette(rev(RColorBrewer::brewer.pal(9, name='RdBu')))(fine).
with.uncertainty	Logical scalar indicating whether to include lower and upper credible interval bounds for the parameter. Default is FALSE.

map	Logical scalar indicating whether to include a map. Default value is FALSE. If TRUE, location must be specified.
state	Logical scalar used when map=TRUE indicating whether the location is a state in the United States (TRUE) or a country (FALSE).
location	Name of region to include in the map. Fed to region in the function <code>ggplot2::map_data</code> .
addthin	Integer indicating the number of saved draws to thin. Default is to not thin any <code>addthin=1</code> . This can save time when the object is from <code>BSTFAfull</code> and <code>parameter='loading'</code> .

Value

A plot of spatially-dependent parameter values for a grid of interpolated locations.

Author(s)

Adam Simpson and Candace Berrett

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(yamat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
map_spatial_param(out, parameter='slope', map=TRUE, state=TRUE, location='utah', fine=50)

## End(Not run)
```

plot_annual	<i>Plot annual/seasonal behavior at a specific location.</i>
-------------	--

Description

Plot annual/seasonal behavior at a specific location.

Usage

```
plot_annual(
  out,
  location,
  add = F,
  years = "one",
  interval = 0.95,
  yrange = NULL,
  new_x = NULL
)
```

Arguments

out	Output from BSTFA or BSTFAfull.
location	Either a single integer indicating the location in the data set to plot or a vector of length 2 providing the longitude and latitude of the new location.
add	Logical scalar indicating whether the annual/seasonal process should be added to the existing plot. Default is FALSE.
years	Either 'one' (indicating to plot just a single year; default) or 'all' (indicating to plot all years in the observed time period).
interval	Numeric value between 0 and 1 specifying the probability of the credible interval.
yrange	Numeric vector of length 2 providing the lower and upper bounds of the y-axis. If NULL (default), the y-axis limits are chosen using the range of the seasonal process and data.
new_x	If the original model included covariates x, include the same covariates for location.

Value

A plot of the annual/seasonal process at location.

Author(s)

Candace Berrett and Adam Simpson

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(ymat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
plot_annual(out, location=1)

## End(Not run)
```

plot_factor

Plot the temporally-dependent factors.

Description

Plot the temporally-dependent factors.

Usage

```
plot_factor(
  out,
  factor = 1,
  together = FALSE,
  include.legend = TRUE,
  type = "mean",
  uncertainty = TRUE,
```

```

    ci.level = c(0.025, 0.975),
    xrange = NULL
  )

```

Arguments

out	Output from BSTFA or BSTFAfull.
factor	Integer or vector of integers specifying which factor(s) to plot.
together	If length(factor)>1, logical scalar specifying whether to plot all factors on a single plot. Default is FALSE.
include.legend	If length(factor)>1 and together=TRUE, a logical scalar specifying whether to include a legend. Default is TRUE.
type	One of mean (default), median, ub, or lb indicating which summary statistic to plot at each location.
uncertainty	Logical scalar indicating whether to include lower and upper credible interval bounds for the parameter. Default is FALSE.
ci.level	A vector of length 2 specifying the quantiles to use for lower and upper bounds for type='lb', type='ub', or uncertainty=TRUE.
xrange	A date vector of length 2 providing the lower and upper bounds of the dates to include in the plot.

Value

A plot of spatially-dependent parameter values for a grid of interpolated locations.

Author(s)

Candace Berrett and Adam Simpson

Examples

```

## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(ymat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
plot_factor(out, factor=1:4, together=TRUE)

## End(Not run)

```

plot_fourier_bases	<i>Visualize fourier bases.</i>
--------------------	---------------------------------

Description

Visualize fourier bases.

Usage

```
plot_fourier_bases(
  coords,
  R,
  fine = 100,
  plot.3d = FALSE,
  freq.lon = 4 * diff(range(coords[, 1])),
  freq.lat = 4 * diff(range(coords[, 2])),
  par.mfrow = c(2, 3)
)
```

Arguments

coords	A matrix of coordinates of observed locations.
R	Integer indicating the number of bases to compute.
fine	Number of grid points to include on both axes. Total grid size will be fine^2 . Default is 100.
plot.3d	Logical scalar indicating whether to plot the bases. Default is FALSE.
freq.lon	Numeric value indicating the frequency to use for the Fourier bases in the longitude direction. Default is $4 \times \text{diff}(\text{range}(\text{coords}[, 1]))$.
freq.lat	Numeric value indicating the frequency to use for the Fourier bases in the latitude direction. Default is $4 \times \text{diff}(\text{range}(\text{coords}[, 2]))$.
par.mfrow	If plot.3d=TRUE, how to divide the plotting window. See <code>help(par)</code> for more details.

Author(s)

Adam Simpson

plot_location	<i>Plot a location's time series of estimated/predicted values.</i>
---------------	---

Description

Plot a location's time series of estimated/predicted values.

Usage

```
plot_location(
  out,
  location,
  new_x = NULL,
  type = "mean",
  par.mfrow = c(1, 1),
  pred.int = TRUE,
  ci.level = c(0.025, 0.975),
  uncertainty = TRUE,
  xrange = NULL,
  truth = FALSE,
  ylim = NULL
)
```

Arguments

out	Output from BSTFA or BSTFAfull.
location	Either a single integer indicating the location in the data set to plot or a vector of length 2 providing the longitude and latitude of the new location.
new_x	If the original model included covariates x, include the same covariates for prediction location.
type	One of mean (default), median, ub, or lb indicating which summary statistic of the predicted values to return.
par.mfrow	A vector of length 2 indicating the number of rows and columns to divide the plotting window. Default is c(1,1).
pred.int	Logical scalar indicating whether the interval should be a posterior predictive interval (default; TRUE) or a posterior credible interval (FALSE).
ci.level	If type='lb' or 'ub', the percentiles for the posterior interval.
uncertainty	Logical scalar indicating whether to plot the uncertainty bounds (TRUE; default) or not.
xrange	A date vector of length 2 providing the lower and upper bounds of the dates to include in the plot.
truth	Logical scalar indicating whether to include the observed measurements (TRUE) or not (default). If TRUE, location must be an integer corresponding to the column of the data matrix for the in-sample prediction location.
ylim	Numeric vector of length 2 providing the lower and upper bounds of the y-axis. If NULL (default), the y-axis limits are chosen using the range of the predictions.

Value

A plot of predicted values for location.

Author(s)

Candace Berrett and Adam Simpson

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(ymat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
plot_location(out, location=1, pred.int=FALSE)

## End(Not run)
```

plot_spatial_param	<i>Plot the spatially-dependent parameter for in-sample locations.</i>
--------------------	--

Description

Plot the spatially-dependent parameter for in-sample locations.

Usage

```
plot_spatial_param(
  out,
  parameter,
  loadings = 1,
  type = "mean",
  ci.level = c(0.025, 0.975),
  yearscale = TRUE,
  color.gradient = (grDevices::colorRampPalette(rev(RColorBrewer::brewer.pal(9, name =
    "RdBu"))))(50)
)
```

Arguments

out	Output from BSTFA or BSTFAfull.
parameter	One of 'slope' (default), 'loading', or 'mean'.
loadings	If parameter='loading', an integer indicating which factor loading to plot.
type	One of mean (default), median, ub, or lb indicating which summary statistic to plot at each location.
ci.level	If type='lb' or 'ub', the percentiles for the posterior interval.
yearscale	If parameter='slope', a logical scalar indicating whether to translate it to a yearly scale (TRUE; default).
color.gradient	The color palette to use for the plot. Default is colorRampPalette(rev(RColorBrewer::brewer.pal(9, name='RdBu')))(50).

Value

A plot of spatially-dependent parameter values for the observed locations.

Author(s)

Adam Simpson and Candace Berrett

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(ymat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
plot_spatial_param(out, parameter='slope')

## End(Not run)
```

plot_trace	<i>Plot trace plots</i>
------------	-------------------------

Description

Plot trace plots

Usage

```
plot_trace(
  out,
  parameter,
  param.range = NULL,
  par.mfrow = c(1, 1),
  density = TRUE
)
```

Arguments

out	Output from BSTFA or BSTFAfull.
parameter	Parameter to plot. See BSTFA and BSTFAfull for parameter names.
param.range	Indices of the named parameter to plot. Default is to plot all relevant parameters.
par.mfrow	Vector of length 2 indicating the number of rows and columns to divide the plotting window.
density	Logical scalar indicating whether to include the density plot of the posterior draws. Default is TRUE.

Value

A plot containing the trace plot (and density plot when `density=TRUE`) of the listed parameters.

Author(s)

Adam Simpson

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(ymat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
plot_trace(out, parameter='beta', param.range=1)

## End(Not run)
```

predictBSTFA

Estimate/predict values of the time series at a specific location.

Description

Estimate/predict values of the time series at a specific location.

Usage

```
predictBSTFA(
  out,
  location = NULL,
  type = "mean",
  ci.level = c(0.025, 0.975),
  new_x = NULL,
  pred.int = TRUE
)
```

Arguments

out	Output from BSTFA or BSTFAfull.
location	Either a single integer indicating the location in the data set to provide predictions or a vector of length 2 providing the longitude and latitude of the new location. If location=NULL (default), the function will return predictions for all in-sample locations.
type	One of all, mean (default), median, ub, or lb indicating which summary statistic of the predicted values to return.
ci.level	If type='lb' or 'ub', the percentiles for the posterior interval.
new_x	If the original model included covariates x, include the same covariates for prediction location.
pred.int	Logical scalar indicating whether to include additional uncertainty for posterior predictive intervals (TRUE; default) or not (posterior draws from the mean of location).

Value

A matrix or vector of estimated/predicted values for location.

Author(s)

Candace Berrett and Adam Simpson

Examples

```
## Not run:
data(utahDataList)
attach(utahDataList)
out <- BSTFA(ymat=TemperatureVals, dates=Dates, coords=Coords, iters=100)
loc1means <- predictBSTFA(out, location=1, pred.int=FALSE)

## End(Not run)
```

utahDataList*Utah Minimum Temperatures*

Description

Zero-centered daily minimum temperatures averaged across 30-day windows from 1919 to 2014 across the US state of Utah; also includes dates, coordinates, and station names.

Usage

```
utahDataList
```

Format

A data set with 4 variables:

TemperatureVals A 1251 by 146 matrix of zero-centered 30-day average daily minimum temperatures from 1912 through 2014. Missing observations are denoted using NA.

Dates A vector of length 1251 and class Date providing the day of each observation in the same order as the rows of TemperatureVals. Note that this package expects observation times to be regularly spaced.

Coords A 142 by 2 data frame containing the longitude (first variable) and latitude (second variable) of measured locations.

Locations A character vector of length 146 containing the station names for measured locations in the same order as the columns of TemperatureVals and the rows of Coords.

Source

<https://climate.usu.edu>

Index

* datasets

utahDataList, [27](#)

bisquare1d, [2](#)

bisquare2d, [2](#)

BSTFA, [3](#)

BSTFAfull, [8](#)

compute_summary, [14](#)

computeLogLik, [14](#)

convergence_diag, [15](#)

defineRegion, [16](#)

makeNewS, [16](#)

makePredS, [17](#)

map_spatial_param, [18](#)

plot_annual, [19](#)

plot_factor, [20](#)

plot_fourier_bases, [21](#)

plot_location, [22](#)

plot_spatial_param, [24](#)

plot_trace, [25](#)

predictBSTFA, [26](#)

utahDataList, [27](#)